Week 1 practice: getting, storing and retrieving data

Student: Saurav
Data: a small *synthetic* course dataset generated in this notebook (no real student data). It mimics the course teaching-data structure: `students.jsonl`, `quiz_attempts.jsonl` and `task_attempts.jsonl`.

What I practise:

1. Creating and reading JSONL files
2. Checking the record grain (what one row represents) and the keys
3. Validating types, ranges and missing values
4. Assigning semantic types (identifier, nominal, ordinal, ratio, datetime)
5. Joining tables safely: row inflation vs aggregating first
6. Keeping raw, prepared and derived data separate
7. Storing tables in SQLite and retrieving them with SQL

1. Create a small synthetic raw dataset (JSONL)

Each line of a JSONL file is one complete JSON record. I write the files to `raw/` and do not change them afterwards.

```
In [1]
import json, sqlite3
from pathlib import Path
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
raw = Path("raw"); raw.mkdir(exist_ok=True)

groups = ["G1", "G2"]
programmes = ["ICT", "Business", "Engineering"]
satisfaction_levels = ["poor", "satisfactory", "good", "excellent"]

students = []
for i in range(1, 11):
    students.append({
        "student_key": f"STU_{i:03d}",
        "group_key": groups[i % 2],
        "programme": programmes[i % 3],
        "satisfaction": satisfaction_levels[rng.integers(0, 4)],
        "registered_at": f"2026-08-{rng.integers(1, 20):02d}",
    })

quiz_attempts = []
for s in students:
    n = int(rng.integers(0, 4))          # some students have no quiz attempts
    for a in range(1, n + 1):
        quiz_attempts.append({
            "attempt_key": f"QA_{len(quiz_attempts)+1:03d}",
            "student_key": s["student_key"],
            "quiz_key": "Q1",
            "attempt": a,
```

```
            "score": round(float(rng.uniform(0.3, 1.0)), 2),
            "submitted_at": f"2026-09-{rng.integers(1, 25):02d}T{rng.integers(8, 22):02d}:00:00",
        })

task_attempts = []
for s in students[:8]:
    for t in ["T1", "T2"]:
        task_attempts.append({
            "student_key": s["student_key"],
            "task_key": t,
            "grade": None if rng.random() < 0.15 else int(rng.integers(1, 6)),
        })

def write_jsonl(path, records):
    with open(path, "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r) + "\n")

write_jsonl(raw / "students.jsonl", students)
write_jsonl(raw / "quiz_attempts.jsonl", quiz_attempts)
write_jsonl(raw / "task_attempts.jsonl", task_attempts)
print(sorted(p.name for p in raw.iterdir()))
```

```
['quiz_attempts.jsonl', 'students.jsonl', 'task_attempts.jsonl']
```

In [2]

```
# Look at the first raw lines: one JSON object per line
print((raw / "quiz_attempts.jsonl").read_text().splitlines()[:3])
```

```
['{"attempt_key": "QA_001", "student_key": "STU_001", "quiz_key": "Q1", "attempt": 1, "score":
0.95, "submitted_at": "2026-09-09T18:00:00"}', '{"attempt_key": "QA_002", "student_key":
"STU_001", "quiz_key": "Q1", "attempt": 2, "score": 0.88, "submitted_at": "2026-09-
16T15:00:00"}', '{"attempt_key": "QA_003", "student_key": "STU_002", "quiz_key": "Q1", "attempt":
1, "score": 0.46, "submitted_at": "2026-09-03T15:00:00"}']
```

2. Read JSONL with pandas and check the grain

`pd.read_json(..., lines=True)` reads line-delimited JSON. For each table I ask: *what does one row represent?*

In [3]

```
students_df = pd.read_json(raw / "students.jsonl", lines=True, dtype=False)
quiz_df     = pd.read_json(raw / "quiz_attempts.jsonl", lines=True, dtype=False)
task_df     = pd.read_json(raw / "task_attempts.jsonl", lines=True, dtype=False)

for name, df in [("students", students_df), ("quiz_attempts", quiz_df), ("task_attempts",
task_df)]:
    print(f"{name:14s} rows={len(df):3d}  unique students={df['student_key'].nunique()}")
```

```
students       rows= 10  unique students=10
quiz_attempts  rows= 19  unique students=8
task_attempts  rows= 16  unique students=8
```

Observation:

- `students` has one row per student (10 rows, 10 unique keys).
- `quiz_attempts` has one row per quiz attempt. The number of rows is larger than the number of unique students, so this is a **one-to-many** relationship.

- `task_attempts` has one row per student × task.

```
In [4]
# Key checks: is student_key unique in students? Do all attempt keys exist in students?
print("student_key unique in students:", students_df["student_key"].is_unique)
print("unknown keys in quiz_attempts:", set(quiz_df["student_key"]) -
set(students_df["student_key"]))
print("students with no quiz attempts:", sorted(set(students_df["student_key"]) -
set(quiz_df["student_key"])))
```

```
student_key unique in students: True
unknown keys in quiz_attempts: set()
students with no quiz attempts: ['STU_006', 'STU_009']
```

3. Validate types, ranges and missing values

```
In [5]
print(quiz_df.dtypes, "\n")
print("score within 0–1:", quiz_df["score"].between(0, 1).all())
print("missing values per column in task_attempts:")
print(task_df.isna().sum())
```

```
attempt_key         str
student_key         str
quiz_key            str
attempt           int64
score           float64
submitted_at        str
dtype: object

score within 0–1: True
missing values per column in task_attempts:
student_key    0
task_key       0
grade          2
dtype: int64
```

Missing ≠ zero. A missing `grade` means the grade is not recorded. It does not mean the student got 0. I keep it as missing (`NaN`) and do not fill it automatically.

Also, a student with **no quiz attempts** is different from a student whose score is unknown. Their quiz count is 0, but their average score is *not applicable*, not 0.

4. Give each variable its semantic type

The technical type from pandas is not enough, so I convert columns to match their meaning.

```
In [6]
prepared_students = students_df.copy()
prepared_students["student_key"] = prepared_students["student_key"].astype("string")      #
identifier
prepared_students["group_key"] = prepared_students["group_key"].astype("category")         #
nominal
prepared_students["programme"] = prepared_students["programme"].astype("category")         #
nominal
prepared_students["satisfaction"] = pd.Categorical(                                          #
ordinal
    prepared_students["satisfaction"], categories=satisfaction_levels, ordered=True)
prepared_students["registered_at"] = pd.to_datetime(prepared_students["registered_at"])    #
```

```
datetime

prepared_quiz = quiz_df.copy()
prepared_quiz["submitted_at"] = pd.to_datetime(prepared_quiz["submitted_at"])
print(prepared_students.dtypes)
```

```
student_key              string
group_key              category
programme              category
satisfaction           category
registered_at    datetime64[us]
dtype: object
```

```
In [7]
# Valid operations depend on meaning:
print("Most common programme (nominal → mode):", prepared_students["programme"].mode()[0])
print("Highest satisfaction (ordinal → max):", prepared_students["satisfaction"].max())
median_code = int(prepared_students["satisfaction"].cat.codes.median())
print("Median satisfaction (ordinal → order-based):", satisfaction_levels[median_code])
print("Days between first and last registration (datetime → difference):",
      (prepared_students["registered_at"].max() - prepared_students["registered_at"].min()).days)
```

```
Most common programme (nominal → mode): Business
Highest satisfaction (ordinal → max): excellent
Median satisfaction (ordinal → order-based): good
Days between first and last registration (datetime → difference): 17
```

I do **not** calculate a mean of `student_key` or of coded programme numbers, because identifiers and nominal categories do not support arithmetic.

5. Joining: row inflation vs aggregating first

```
In [8]
joined = prepared_students.merge(prepared_quiz, on="student_key", how="inner")
print("students rows:", len(prepared_students), "| after join rows:", len(joined))
joined[["student_key", "group_key", "attempt", "score"]].head(6)
```

```
students rows: 10 | after join rows: 19
```

	student_key	group_key	attempt	score
0	STU_001	G2	1	0.95
1	STU_001	G2	2	0.88
2	STU_002	G1	1	0.46
3	STU_003	G2	1	0.88
4	STU_003	G2	2	0.83
5	STU_003	G2	3	0.98

After the join, **one row = one quiz attempt**, not one student. Students with several attempts are repeated. With an inner join, students with no attempts disappear.

If I count students per group from `joined`, the result is wrong:

```
In [9]
print("WRONG — rows per group after join:\n", joined.groupby("group_key", observed=True).size(),
"\n")
```

```
print("RIGHT — students per group:\n", prepared_students.groupby("group_key",
observed=True).size())
```

```
WRONG — rows per group after join:
 group_key
G1     9
G2    10
dtype: int64

RIGHT — students per group:
 group_key
G1    5
G2    5
dtype: int64
```

In [10]

```
# Correct student-level table: aggregate attempts per student first, then LEFT join
quiz_per_student = (prepared_quiz.groupby("student_key")
                    .agg(quiz_attempts=("attempt_key", "count"),
                         average_score=("score", "mean"),
                         last_submitted=("submitted_at", "max"))
                    .reset_index())

student_level = prepared_students.merge(quiz_per_student, on="student_key", how="left")
student_level["quiz_attempts"] = student_level["quiz_attempts"].fillna(0).astype(int)  # no rows
→ 0 attempts (justified)
# average_score stays NaN for students without attempts (not applicable, not 0)
print("rows:", len(student_level), "(still one row per student)")
student_level[["student_key", "group_key", "quiz_attempts", "average_score"]]
```

```
rows: 10 (still one row per student)
```

	student_key	group_key	quiz_attempts	average_score
0	STU_001	G2	2	0.915000
1	STU_002	G1	1	0.460000
2	STU_003	G2	3	0.896667
3	STU_004	G1	3	0.626667
4	STU_005	G2	3	0.656667
5	STU_006	G1	0	NaN
6	STU_007	G2	2	0.535000
7	STU_008	G1	3	0.746667
8	STU_009	G2	0	NaN
9	STU_010	G1	2	0.355000

`quiz_attempts`, `average_score` and `last_submitted` are **derived** variables: they are calculated from the raw attempts.

Filling `quiz_attempts` with 0 is justified, because a student with no attempt rows really made 0 attempts. Filling `average_score` with 0 would **not** be justified.

6. Keep raw, prepared and derived data separate

In [11]

```
prepared = Path("prepared"); prepared.mkdir(exist_ok=True)
student_level.to_csv(prepared / "student_level.csv", index=False)
```

```
print("raw/:", sorted(p.name for p in raw.iterdir()))
print("prepared/:", sorted(p.name for p in prepared.iterdir()))
```

```
raw/: ['quiz_attempts.jsonl', 'students.jsonl', 'task_attempts.jsonl']
prepared/: ['student_level.csv']
```

7. Store in SQLite and retrieve with SQL

SQLite keeps the database in one file (`course-data.db`). pandas writes the tables with `to_sql()` and reads query results with `read_sql()`.

```
In [12]
con = sqlite3.connect("course-data.db")
students_df.to_sql("students", con, if_exists="replace", index=False)
quiz_df.to_sql("quiz_attempts", con, if_exists="replace", index=False)
task_df.to_sql("task_attempts", con, if_exists="replace", index=False)
print(pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", con))
```

```
           name
0      students
1  quiz_attempts
2  task_attempts
```

```
In [13]
# SELECT + WHERE: attempts with score above 0.7
pd.read_sql("SELECT student_key, attempt, score FROM quiz_attempts WHERE score > 0.7 ORDER BY
score DESC", con)
```

	student_key	attempt	score
0	STU_003	3	0.98
1	STU_005	1	0.98
2	STU_001	1	0.95
3	STU_001	2	0.88
4	STU_003	1	0.88
5	STU_008	2	0.88
6	STU_004	1	0.84
7	STU_003	2	0.83
8	STU_008	1	0.79

```
In [14]
# GROUP BY: average score per student
pd.read_sql("""
    SELECT student_key, COUNT(*) AS attempts, ROUND(AVG(score), 2) AS avg_score
    FROM quiz_attempts
    GROUP BY student_key
""", con)
```

	student_key	attempts	avg_score
0	STU_001	2	0.92
1	STU_002	1	0.46
2	STU_003	3	0.90
3	STU_004	3	0.63
4	STU_005	3	0.66

	student_key	attempts	avg_score
5	STU_007	2	0.54
6	STU_008	3	0.75
7	STU_010	2	0.35

In [15]
```
# JOIN + WHERE: all quiz attempts of students in group G1
pd.read_sql("""
    SELECT s.student_key, s.group_key, q.attempt, q.score
    FROM students s
    JOIN quiz_attempts q ON s.student_key = q.student_key
    WHERE s.group_key = 'G1'
""", con)
```

	student_key	group_key	attempt	score
0	STU_002	G1	1	0.46
1	STU_004	G1	1	0.84
2	STU_004	G1	2	0.63
3	STU_004	G1	3	0.41
4	STU_008	G1	1	0.79
5	STU_008	G1	2	0.88
6	STU_008	G1	3	0.57
7	STU_010	G1	1	0.40
8	STU_010	G1	2	0.31

In [16]
```
# LEFT JOIN keeps students without attempts; aggregating keeps one row per student
pd.read_sql("""
    SELECT s.student_key, s.group_key, COUNT(q.attempt_key) AS attempts, ROUND(AVG(q.score), 2)
AS avg_score
    FROM students s
    LEFT JOIN quiz_attempts q ON s.student_key = q.student_key
    GROUP BY s.student_key, s.group_key
""", con)
```

	student_key	group_key	attempts	avg_score
0	STU_001	G2	2	0.92
1	STU_002	G1	1	0.46
2	STU_003	G2	3	0.90
3	STU_004	G1	3	0.63
4	STU_005	G2	3	0.66
5	STU_006	G1	0	NaN
6	STU_007	G2	2	0.54
7	STU_008	G1	3	0.75
8	STU_009	G2	0	NaN
9	STU_010	G1	2	0.35

In [17]
```
con.close()
```

8. What I learned

- Data arrived as **several JSONL files**, not one table. Each file has its own grain.
- `student_key` connects the tables. It is an **identifier**, not a number to calculate with.
- A join from a one-row-per-student table to an attempts table **changes the grain** and can inflate counts. Aggregating first keeps one row per student.
- **Missing values need a reason**: 0 attempts is a real zero, but an average score for a student with no attempts is not applicable.
- The pandas type does not decide the analysis. I set categorical, ordered categorical and datetime types to match what each variable means.
- I kept `raw/` unchanged and wrote prepared data to `prepared/` and to SQLite, where SQL (`SELECT`, `WHERE`, `GROUP BY`, `JOIN`) retrieved exactly what I needed.